

Optimization

CSC 5666: *Advanced Machine Learning*, Fall 2026

Professor Austin P. Wright: Course Notes

Abstract

Introduction to the foundational concepts in optimization theory that are used in the context of probabilistic inference and machine learning. Closely follows Chapter 6 of Murphy's textbook.

I Introduction to Optimization

So far in this course we have introduced many mathematical constructs that represent process we wish to model. However when it comes to actually constructing programs or models that have the properties we want, this most often requires some form of optimization. In extreme generality, this means writing problems in the form

$$\theta^* \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) \quad (1)$$

where $\mathcal{L} : \Theta \rightarrow \mathbb{R}$ is the objective or loss function and Θ is the parameter space we optimize over. What we have yet to discuss is *how* to actually construct θ to where those objectives want it.

Optimization theory is a rich field unto itself, and I will assume some minimal prior familiarity with it. What follows refreshes the parts this course leans on, plus a few topics that matter in the probabilistic and Bayesian setting.

In particular we will simply take as a starting point the existence of some class of differentiable functions about which we can take gradients. In reality there is a huge amount of work involved in the construction of such functions, and they can take the form of restricted classes of computation graph such as Pytorch¹, or can be more complex *differentiable programs*. Systems such as JAX², and Dex³ are examples of highly general purpose differentiable programming systems built around a pure functional style.

II Stochastic Gradient Descent

In this class the majority of objects about which we are trying to optimize are probabilistic. Thus the optimization objectives we care about will frequently take the form:

$$\mathcal{L}(\theta) = \mathbb{E}_{q_\theta(z)} [\tilde{\mathcal{L}}(\theta, z)] \quad (2)$$

where θ are parameters over which we want to optimize, and z is a random variable, modeling something like external noise.

Contents

I	Introduction to Optimization	1
II	Stochastic Gradient Descent	1
III	Natural Gradient Descent	5
IV	Additional Optimization Methods	7

Textbook: Chapter 6, pp. 261–342

¹<https://pytorch.org/>

²<https://docs.jax.dev/en/latest/index.html>

³<https://google-research.github.io/dex-lang/>

Textbook: §6.3, pp. 271–279

If we then have a way of calculating an estimate of the gradient of the objective function, we can use the procedure of gradient descent to improve our estimates of θ iteratively using a *learning rate*, α .

$$\theta_{t+1} = \theta_t + \alpha_t (\nabla_{\theta} \mathcal{L}(\theta) |_{\theta_t}) \quad (3)$$

This is *stochastic gradient descent (SGD)*, differentiated as stochastic because the gradient being calculated is in face an *expectation* over some random variables.

Note that we actually index the learning rate as a function of the iteration step t . This is because in general we want to lower the learning rate over time, as big updates make sense early on in the process where we are far away from an optimal solution, but smaller steps are needed to fine tune closer to an optimum. In fact in general it can be shown that any learning rate schedule that meets the Robbins-Monro conditions will reach convergence in SGD.

$$\alpha_t \rightarrow 0 \text{ and } \frac{\sum_t 1^\infty \alpha_t^2}{\sum_t 1^\infty \alpha_t} \rightarrow 0 \quad (4)$$

Common schedules that are used (or sometimes combined) are:

- Piecewise constant: $\alpha_t = \alpha_i$ if $t_i \leq t < t_{i+1}$
- Exponential decay: $\alpha_t = \alpha_0 e^{-\lambda t}$
- Polynomial decay: $\alpha_t = \alpha_0 (b \cdot t + 1)^{-a}$
- Reduce on plateau: Estimate if loss has stopped decreasing enough over some time horizon, and if so reduce α

In addition to all of these methods there is a huge array of tricks and variants on the basic SGD process. Any of the libraries you are likely to deal with will have support for many of them, and in practice the different learning rate values (or schedule hyperparameters) are determined through pure empirical trial and error for any given optimization.

Discussion

Above we motivated decaying α_t by saying that large updates make sense early and small ones late. However, some research has shown that you can instead hold α fixed and grow the batch size on the same schedule, and obtain essentially the same training curve. Without writing any algebra, say what quantity the two procedures are holding in common, and what that implies about which of the two knobs is the real one. Then give one practical reason you might still prefer decay, and one reason you might prefer growth.⁴

⁴ *Hint:* A mini-batch is defined as using a small sample of data-point for the SGD expectation which otherwise is over the full dataset. If we think of the mini-batch loss then as a noisy estimate of the overall loss, what changes about this estimate when changing the size of the mini-batch?

Reparametrization Trick

Textbook: §6.3.5, pp. 275–276

Consider now the case where the stochasticity we are optimizing over itself depends on the parameters we are optimizing. This happens frequently in Reinforcement learning where z may be actions sampled from a policy we are training, or more relevantly for us z may be a *latent variable* sampled from an inference network q_θ which occurs in stochastic variational inference. In this case we have the gradient:

$$\begin{aligned}\nabla_\theta \mathbb{E}_{q_\theta(z)} [\tilde{\mathcal{L}}(\theta, z)] &= \nabla_\theta \int \tilde{\mathcal{L}}(\theta, z) q_\theta(z) dz \\ &= \int \nabla_\theta \tilde{\mathcal{L}}(\theta, z) q_\theta(z) dz \\ &= \int [\nabla_\theta \tilde{\mathcal{L}}(\theta, z)] q_\theta(z) dz + \int \tilde{\mathcal{L}}(\theta, z) [\nabla_\theta q_\theta(z)] dz \quad (5)\end{aligned}$$

While the first term can be approximated by Monte-Carlo sampling, the second term cannot. Thus in order to estimate the gradient we need to use something called the *reparametrization trick*.⁵

For this we need to be able to meet two additional requirements:

1. $\tilde{\mathcal{L}}(\theta, z)$ must be differentiable with respect to z
2. We must be able to sample from $q_\theta(z)$ by first sampling ϵ from some noise distribution that is *independent of* θ and then transforming to z using a deterministic and differentiable function $z = g(\theta, \epsilon)$. Most frequently this will be something like producing a sample $z \sim \mathcal{N}(\mu, \sigma^2)$ by first sampling $\epsilon \sim \mathcal{N}(0, 1)$ and then calculating $z = \mu + \sigma\epsilon$ (where here $\theta = (\mu, \sigma)$).

Doing this allows us to rewrite the objective

$$\mathcal{L}(\theta) = \mathbb{E}_{q_\theta(z)} [\tilde{\mathcal{L}}(\theta, z)] = \mathbb{E}_{q_0(\epsilon)} [\tilde{\mathcal{L}}(\theta, g(\theta, \epsilon))], \quad (6)$$

Since now the expectation is over a distribution *independent of the parameters* θ , we can push the gradient inside the expectation and approximate with Monte-Carlo.

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{q_0(\epsilon)} [\nabla_\theta \tilde{\mathcal{L}}(\theta, g(\theta, \epsilon))] \approx \frac{1}{S} \sum_{s=1}^S \nabla_\theta \tilde{\mathcal{L}}(\theta, g(\theta, \epsilon_s)). \quad (7)$$

Discussion

Consider the point in the derivation above where moving the gradient inside the expectation became legal. What would have gone wrong if we just applied the same step to the first form? Then answer the question that should be bothering you: Since the reparametrized objective is the same objective, taking the same value for every θ , how can rewriting an expression change which gradients we are able to compute?

⁵While there are other tricks we could use such as Score Function Estimation or REINFORCE, the reparametrization trick tends in practise to be lower variance and thus preferred if we can reparametrize the problem appropriately.

The Gumbel-Softmax Relaxation

The reparametrization trick is very powerful and useful, however not all distributions can be reparametrized. Most notably perhaps are discrete variables which we often want to be able to encode in a latent representation. In such cases we can *relax* the discrete variables into continuous variables in a way that allows the trick to be used.

Consider a one-hot vector d with K bits, that is $d \in \{0, 1\}^K$ and $\sum_k d_k = 1$. This vector represents a categorical variable with K states. Then let $P(d) = \text{Cat}(d|\pi)$ where $\pi_k = P(d_k = 1)$ ⁶ define the probability distribution of the variable. We can sample a one-hot vector d from this distribution by computing

$$d = \text{onehot}\left(\underset{k}{\operatorname{argmax}}[\varepsilon_k + \log(\pi_k)]\right) \quad (8)$$

Where $\varepsilon_k \sim \text{Gumbel}(0, 1)$ is sampled from the Gumbel distribution. This gives us a reparametrization, however it is not a very useful one.

Discussion

Before moving on, what is the issue with this reparametrization?

The big problem with the representation is that the argmax function is useless for gradient based optimization! It is zero everywhere except for on the borders of transitions from class to class, at which point it is undefined. Thus we will replace the argmax with *softmax*, and the vector d with a continuous relaxation x over the K dimensional simplex⁷ allows us to write:

$$x_k = \frac{\exp((\log(\pi_k) + \varepsilon_k)/\tau)}{\sum_{k'=1}^K \exp((\log(\pi_{k'}) + \varepsilon_{k'})/\tau)} \quad (9)$$

Where $\tau > 0$ is the *temperature* hyperparameter. This is called the *Gumbel-softmax distribution*, which smoothly approximates the discrete distribution as $\tau \rightarrow 0$, and thus allows us to replace $f(d)$ with $f(x)$ and take gradients with respect to x .

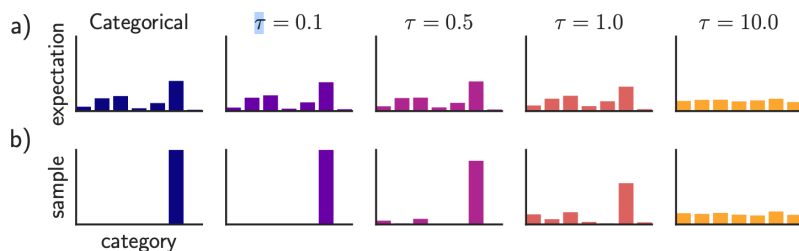


Figure 1: Illustration of the Gumbel-softmax distribution with $K = 7$ states at different temperatures τ . The top row shows $\mathbb{E}[z]$, and the bottom row shows samples $z \sim \text{GumbelSoftmax}(\pi, \tau)$. The left column shows a discrete (categorical) distribution, which always produces one-hot samples. From Textbook Figure 6.3.

Textbook: §6.3.6, pp. 277–278

⁶We could also define the parametrization of the distribution for un-normalized values of π , and calculate the normalization when needed.

⁷The simplex is just defined as the continuous multi-dimensional surface over the same space as the one-hot vector.

$$x \in \left\{ x \in \mathbb{R}^K \mid x_k \in [0, 1], \sum_{k=1}^K x_k = 1 \right\}$$

Discussion

For a single categorical latent with $K = 7$ states, as in the figure, the objective $\mathbb{E}[\hat{\mathcal{L}}]$ is a sum of seven terms whose gradient can be computed exactly with no relaxation. If this is the case, why is the relaxation nonetheless the standard tool. What has to be true of the model for exact enumeration to stop being an option?

III Natural Gradient Descent

So far when doing optimization we have been performing updates in terms of *parameters*, θ , of a distribution. However, it is important to note that differences in parameters not the same thing as differences in the distributions themselves.

Textbook: §6.4

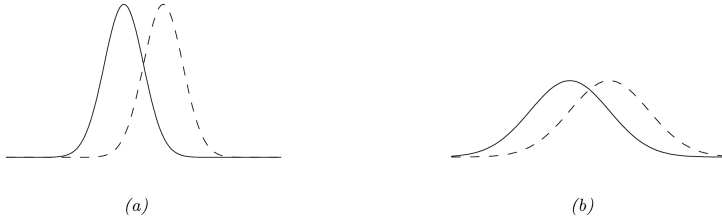


Figure 2: Shifting a Gaussian's mean by the same amount δ (solid to dotted) when the variance is (a) small or (b) large. The Euclidean distance moved in parameter space is identical; the KL divergence, and so the change in information, is not. Reproduced from Murphy, Textbook Figure 6.5, p. 280.

Natural Gradient Descent (NGD) is defined in terms of the distance between two probability distributions, and thus in terms of KL divergence.

Let us say we have a conditional distribution $p_\theta(y|x)$ for which we want to perturb the parameters θ , the divergence is thus given by⁸

$$D_{\text{KL}}(p_\theta \| p_{\theta+\delta}) \approx \frac{1}{2} \delta^T F_x(\theta) \delta, \quad (10)$$

where F_x is the *Fisher information matrix*.

$$\begin{aligned} F_x(\theta) &= -\mathbb{E}_{p_\theta(y|x)} [\nabla_\theta^2 \log p_\theta(y|x)] \\ &= \mathbb{E}_{p_\theta(y|x)} [(\nabla_\theta \log p_\theta(y|x))(\nabla_\theta \log p_\theta)^T] \end{aligned} \quad (11)$$

And we can compute the average KL between the current and update distributions using $\frac{1}{2} \delta^T F(\theta) \delta$ where F is the average FIM.

$$F(\theta) = \mathbb{E}_{p_{\mathcal{D}}(x)} [F_x(\theta)] \quad (12)$$

For NGD we then use the inverse FIM as a *preconditioning matrix* for updates of the form

$$\begin{aligned} \theta_{t+1} &= \theta_t - \alpha_t F(\theta_t)^{-1} \nabla_\theta \mathcal{L}(\theta_t) \\ &= \theta_t - \alpha_t \tilde{\nabla}_\theta \mathcal{L}(\theta_t) \end{aligned} \quad (13)$$

Where $\tilde{\nabla}_\theta$ is called the *natural gradient*.

Natural gradient descent can be much more robust to noise when compared to standard gradient descent, especially the noise introduced by mini-batch approximations where we can recompute F online per mini-batch. Additionally, since NGD

⁸ If you remember what we discussed about information theory, this move should set off some alarm bells. We heavily emphasized that KL is neither a distance nor symmetric, and here it behaves like a metric. In this case this is ok because the asymmetry is third order: to second order $D_{\text{KL}}(p_\theta \| p_{\theta+\delta})$ and $D_{\text{KL}}(p_{\theta+\delta} \| p_\theta)$ are the same quantity, $\frac{1}{2} \delta^T F \delta$, for small δ . The local geometry is symmetric even though the global divergence is not, which is why we did not have to say which way round the arguments went.

updates parameters in a way that *matters more for the predictions generated from the distribution*, we can sometimes get away with larger learning rates at least in the initial steps of the optimization, and somewhat avoid getting stuck in plateaus. Finally, since NGD is invariant to how the distribution is parameterized, which is useful when we are representing distributions using complex models such as neural networks.

Approximating the Natural Gradient

The main cost of NGD is in the calculation of the inverse of the Fisher Information Matrix. Frequently assumptions about the structure of F are made in order to speed up this process, such as using a diagonal approximation or some other kind of factorization.

A simpler approach is to approximate the FIM by replacing the model's distribution with the empirical distribution, which allows us to calculate the *empirical Fisher*

$$F \approx \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \nabla \log(p(y|x, \theta)) \nabla \log(p(y|x, \theta))^\top \quad (14)$$

While this is a widely used approximation since it is simple to compute⁹, the empirical Fisher does not work quite as well as the true Fisher. In particular in “plateau” regions the empirical Fisher becomes singular, while the true Fisher marginalizes out y which allows it to detect small changes in the output by changing the parameters. Thus one of the main benefits of NGD is restricted to the more expensive full case.

⁹AdaGrad uses a similar approach but including a momentum component.

Discussion

1. Despite the fact that NGD has a bevy of theoretical benefits, it is not used very frequently in practice. Why is this the case? Consider how much does NGD actually cost, and which of the benefits survive the approximations required to overcome this cost.
2. The figure at the top of this section shows how the μ parameter space affects distributions. Redraw the argument for σ : for a narrow Gaussian and a wide one, which is more changed in distribution terms, for a constant $\Delta\sigma$? Sketch what a preconditioner therefore has to do to the two coordinates, and thus describe something F can do here that no choice of per-parameter learning rates could do.

IV Additional Optimization Methods

Beyond gradient descent, we sometimes want to be able to perform optimization in non-differentiable contexts. These will be considered outside of the scope of this course, however they are extremely interesting and powerful and may be useful to you in your project or other work, so we will briefly summarize some of the most valuable methods worth exploring. Many of these methods become useful in the *meta-learning* process of choosing appropriate modeling choices and hyperparameters for which we cannot directly optimize.

Bayesian Optimization

If we have an optimization problem for which it is very expensive to generate samples, and for which we cannot calculate gradients (a *black box*), Bayesian Optimization (BayesOpt) provides a very powerful option.

Since the true objective function f may be expensive to evaluate (such as running a simulation or training a model), our goal is to make as few *queries* of the *oracle* f as possible. In order to do this we construct a *surrogate function* based on the data collected so far for which we can decide which point to query next. This task has an inherent tradeoff between picking a point x for which we think $f(x)$ will be good¹⁰, and picking points about which we are *uncertain*.¹¹ In the discrete case this becomes similar to the *multi-arm bandit* problem. The most frequent surrogate function choice in BayesOpt is a Gaussian process¹², as it gives us very high flexibility and good models of uncertainty. It also allows us to compute the *acquisition function* to determine the next sample point x_n with the highest expected utility, where our choice of utility model will determine where the optimization lands on exploration vs exploitation.

¹⁰ In the BayesOpt literature the convention is to actually be *maximizing* f .

¹¹ This is yet another instance of the classic, *exploration exploitation* dilemma.

¹² This is also a class of model that we will not discuss in depth but it is very much worth learning for its own sake as they support closed form Bayesian updating and work well in cases of small data.

Algorithm 1: Bayesian Optimization

```

1: ▷ Given acquisition function,  $\alpha$ 
2: ▷ Collect initial dataset,  $\mathcal{D}_0 = \{(x_i, y_i)\}$ 
3: ▷ Initialize model by computing,  $p(f|\mathcal{D}_0)$ 
4: for  $n = 1, 2, \dots$  until convergence do
5:   ▷ Choose next query point,  $x_{n+1} = \operatorname{argmax}_x \alpha(x; \mathcal{D}_n)$ 
6:   ▷ Measure function value,  $y_{n+1} = f(x_{n+1}) + \varepsilon_n$ 
7:   ▷ Augment dataset,  $\mathcal{D}_{n+1} = \{(y_{n+1}, x_{n+1})\} \cup \mathcal{D}_n$ 
8:   ▷ Update model by computing,  $p(f | \mathcal{D}_{n+1})$ 
9: end
10: return  $\max \mathcal{D}_{\text{end}}$ 

```

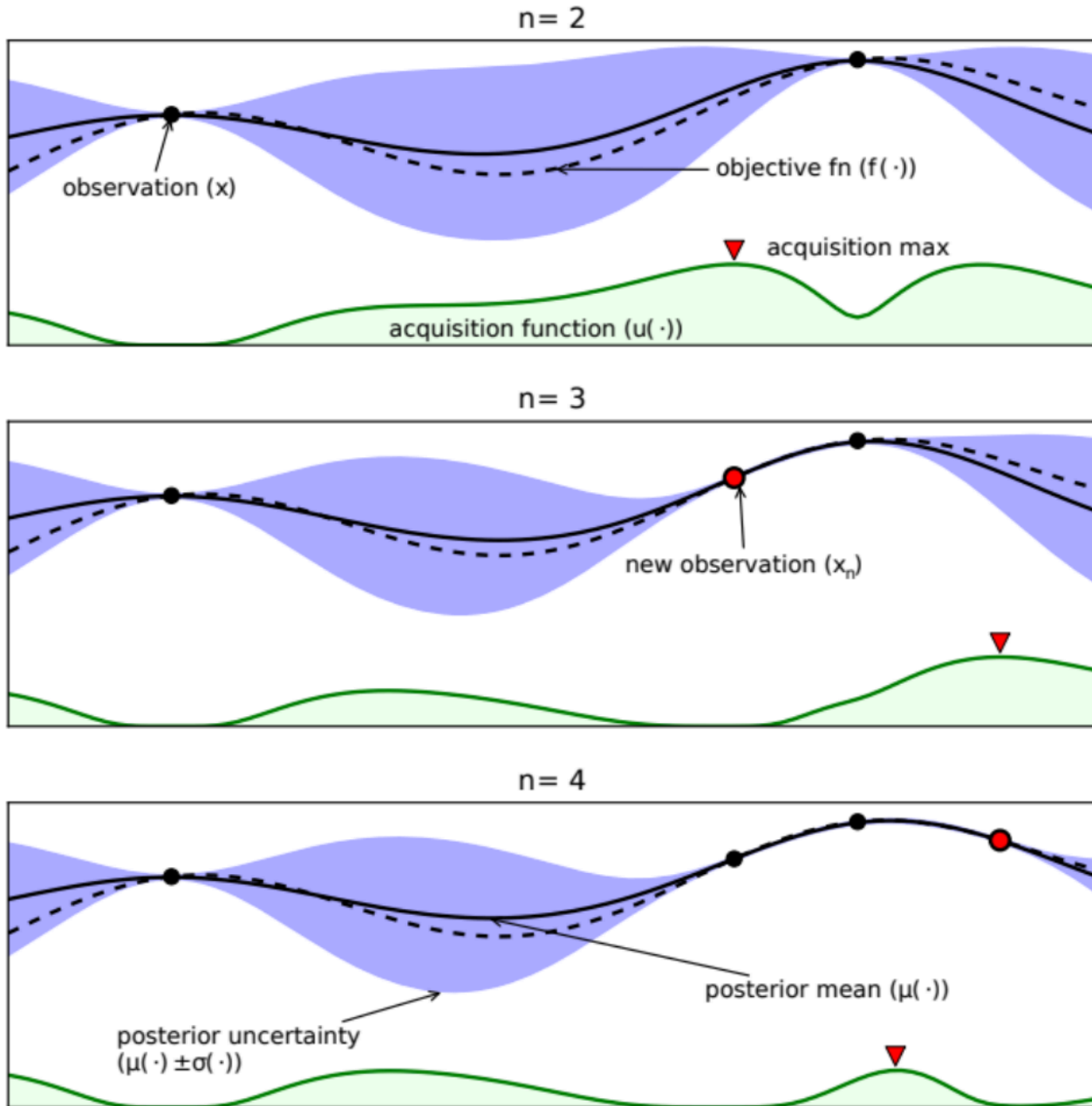


Figure 3: Illustration of sequential Bayesian optimization over three iterations. The rows correspond to a training set of size $t = 2, 3, 4$. The dotted black line is the true, but unknown, function $f(x)$. The solid black line is the posterior mean, $\mu(x)$. The shaded blue intervals are the 95% credible interval derived from $\mu(x)$ and $\sigma(x)$. The solid black dots correspond to points whose function value has already been computed, i.e., x_n for which $f(x_n)$ is known. The green curve at the bottom is the acquisition function. The red dot is the proposed next point to query, which is the maximum of the acquisition function. From Textbook Figure 6.10

Evolutionary Algorithms

If we do not have the restriction that computing the optimization is expensive, but keep the restriction that it is not differentiable, one interesting method is to use an *Evolutionary Algorithm (EA)*, where we maintain a set or *population* of candidates at each point in time, which we try to improve (locally for each candidate) at each step. In many ways this is a more efficient form of just guessing with local improvements, since it will explore more of the search space at once, and information from different members of the population can be shared.

Since, as you might expect, evolutionary algorithms take inspiration from biology, they often also borrow much of the same terminology. Thus we have:

- The *fitness* of a member of the population is the value of the objective function (possibly normalized across population members)
- The members of the population at step $t + 1$ are called the *offspring* which are created by random *recombination* of *parents* at step t and application of some randomness as a *mutation*.
- The procedure that determines the parents that are chosen to *reproduce* is called the *selection function*

Different forms of EA generally are differentiated by their selection functions and recombination and mutation procedures. Examples include *genetic algorithms* which use a recombination and mutation method based on *crossover* of candidate vector representations, and *genetic programming* which uses a tree structured representation.

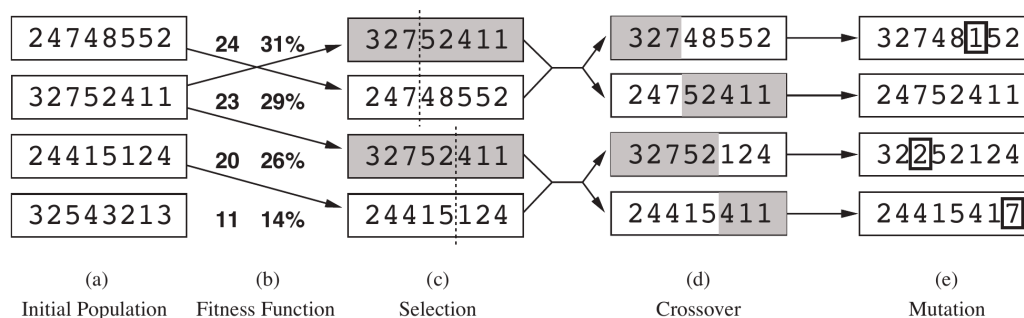


Figure 4: Illustration of a genetic algorithm applied to the 8-queens problem. (a) Initial population of 4 strings. (b) We rank the members of the population by fitness, and then compute their probability of mating. Here the integer numbers represent the number of nonattacking pairs of queens. From Textbook Figure 6.13